How Much Does Noise Affect Fuzzy Models? A Comparison with Shallow Nets

Alexandre Horta Baptista (100514) Saúl Sandinha Gomes (100266)

October 21, 2025

1 Introduction

Every real dataset comes with noise—from sensors, compression, or missing pixels. We compare two simple models on MNIST: an interpretable fuzzy classifier (RBF rules with a linear readout) and a shallow neural network (SNN). Using a single severity scale that makes heterogeneous corruptions comparable, we ask two questions: how quickly does each model break as noise grows, and can lightweight, classical preprocessing meaningfully help?

Steps

- A standardized noise severity scale that maps heterogeneous noise parameters (e.g., Gaussian , JPEG compression, salt and pepper) into $[0, 1]$.
- A stronger yet interpretable fuzzy baseline: multi-rule (clustered) Gaussian membership with a logistic-regression readout.
- A thorough comparison across many noise types and preprocessing pipelines.
- Public code and reproducibility checklist.

2 Dataset

We use MNIST [**?**] (28×28 grayscale digits 0–9). Data are normalized to $[0, 1]$. We split into train/validation/test, with 90% validation, 10% test.

3 Methods

3.1 Noise processes and standardized severity

We inject these noises: Gaussian, uniform, speckle, shot,Poisson , salt & pepper, dropout (pixel masking), motion blur, JPEG compression, anisotropic Gaussian.

We compare at severities $s \in [0, 1]$ and map to type-specific parameters:

$$\text{Gaussian/Uniform/Speckle/S\&P/Dropout}: \quad \theta(s) = 0.05 + 0.45\, s, \tag{1}$$

$$\text{Shot}: \quad \theta(s) = 0.5 + 4.5\, s, \tag{2}$$

$$\text{Motion blur kernel}: \quad k(s) = \text{odd}(\text{round}(3 + 6\, s))\,, \tag{3}$$

$$\text{JPEG quality}: \quad q(s) = \text{round}(95 - 75\, s), \tag{4}$$

$$\text{Quantization bits}: \quad b(s) = \text{round}(8 - 6\, s), \tag{5}$$

$$\text{Anisotropic std}_x: \quad \sigma_x(s) = 0.1 + 0.4\, s, \quad \sigma_y = \sigma_x/4. \tag{6}$$

This yields a comparable x-axis across noise types in all plots.

3.2 Preprocessing

We evaluate: median filter, Gaussian blur, Non-Local Means (NL-means), Total Variation (TV) denoising, wavelet shrinkage, Wiener filtering. We ran each preprocessing method one type for each noise type, and picked the best based on the results. Parameters used are in Table 1.

Table 1: Preprocessing modes and defaults (used unless otherwise stated).

Mode	Main parameters
Median	radius=1
Gaussian blur	$\sigma = 0.8$
NL-means	$h = 0.8$, patch_size=3, patch_distance=5
TV (Chambolle)	weight=0.08
Wavelet	soft threshold, rescale_sigma=true
Wiener	window=3
Auto	per-noise heuristic (median for S&P, Wiener for motion blur, NL-means for JPEG, etc.)

3.3 Fuzzy model (RBF rules + linear readout)

Let $Z \in \mathbb{R}^D$ be PCA features (we use $D = 40$ by default; we also report 64D). We learn one rule per class (with ten classes, one for each number). Rule r has center m_r and diagonal width σ_r. Membership for a sample z:

$$\mu_r(z) \;=\; \exp\!\left(-\tfrac{1}{2}\sum_{d=1}^{D}\left(\frac{z_d - m_{r,d}}{\sigma_{r,d}}\right)^2\right), \qquad \phi(z) = \big[\mu_1(z), \ldots, \mu_R(z)\big]. \tag{7}$$

We normalize ϕ. A multinomial logistic regression predicts class probabilities:

$$p(y = k \mid z) \;=\; \frac{\exp(w_k^\top \phi(z))}{\sum_{k'} \exp(w_{k'}^\top \phi(z))}. \tag{8}$$

This preserves interpretability: rules are localized fuzzy regions, and the linear weights show how each rule votes.

3.4 Shallow neural network (SNN)

A one-hidden-layer MLP with hidden size H (default H=64), ReLU, cross entropy. We train either on raw pixels or PCA features depending on the setting (`--snn-space`).

3.5 Training and evaluation

We fit PCA on clean training data only, train both models on clean data, and evaluate on test data corrupted at severities $s \in \{0, 0.25, 0.5, 0.75, 1\}$, with and without preprocessing. Metrics: accuracy and the full confusion matrix (appendix). We report global curves across all noise types and per-noise curves, with and without preprocessing.

Robustness-oriented training variant (final experiment.py). Besides the default setting where models are trained on clean data, we also include a separate training script that targets stronger test-time robustness. It augments the training set with random noise drawn from the same severity grid, and can apply the same auto preprocessing to high-severity augmented samples, if they have a high enough level of severity. PCA is fitted on the augmented distribution for consistency. At evaluation, we keep the same validation-gated auto policy. This

script elaborates on whether preprocessing helps primarily as a train-time prior or as a test-time post-processing step.

4 Results

4.1 Clean performance

On clean test data ($s = 0$), the SNN reaches **97.33%** accuracy; the fuzzy model reaches **86.90%** with $K =$ one rule per class and σ-scale=best (selected on validation).

4.2 Global robustness curves

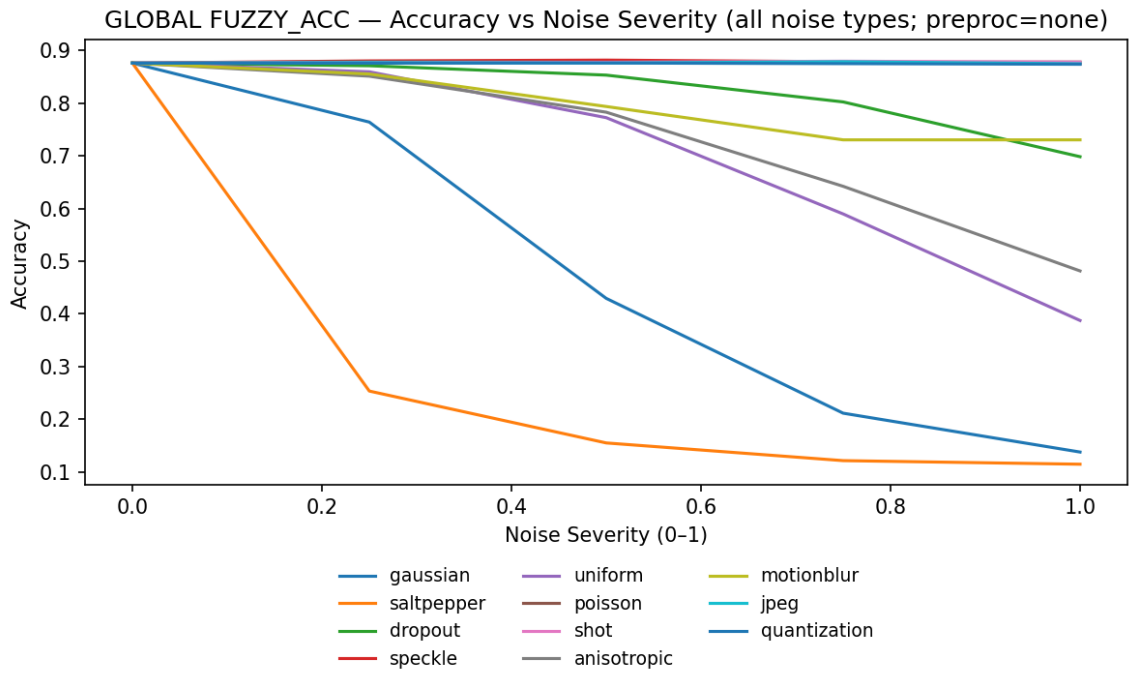

Figure 1: Fuzzy (RBF+LR): accuracy vs. standardized noise severity across all noise types (*no preprocessing*).

Fuzzy, no preprocessing. Figure 1 shows the accuracy of the fuzzy classifier under increasing noise severity without any preprocessing. We observe four trends. First, smooth or low-frequency noise (uniform, Poisson, speckle, shot, anisotropic) causes gradual degradation, since the underlying digit structure is preserved and Gaussian memberships still align with class centroids. Second, impulsive noise (salt–pepper) produces an abrupt collapse: sparse pixel flips destroy local stroke continuity so the centroid rules no longer represent the input distribution. Third, Gaussian noise is harmful but less catastrophic than impulsive corruptions, as edges blur but are not destroyed. Finally, spatially correlated distortions such as motion blur, JPEG, and quantization degrade more slowly at low severities but eventually remove high-frequency detail needed for stable membership assignment.

In conclusion, the baseline fuzzy model is naturally robust to smooth distortions but brittle to discontinuous/sparse corruptions. This justifies introducing preprocessing selectively (rather than always-on).

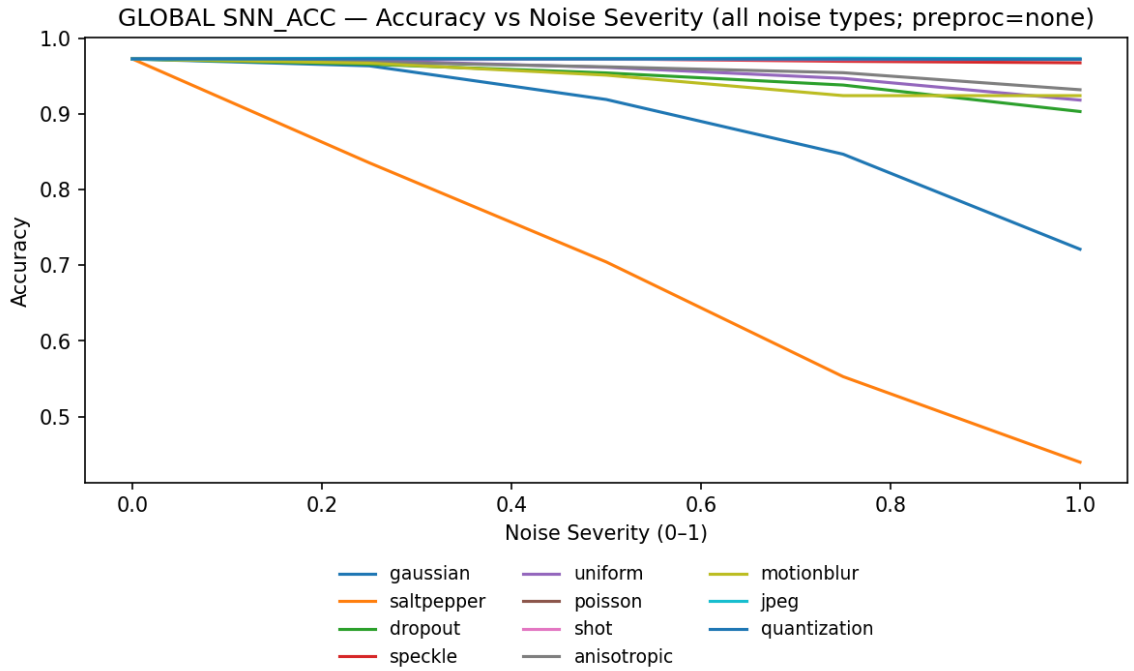

Figure 2: SNN: accuracy vs. standardized noise severity across all noise types (*no preprocessing*).

SNN, no preprocessing Figure 2 reports the SNN without preprocessing. Compared to fuzzy, the SNN is substantially more resilient in the low–to–moderate region: most noise types remain above 95% up to severity 0.25, indicating that the learned representation (PCA+SNN) already absorbs much of the distortion structure. Salt–pepper remains the dominant failure case, but the SNN degrades more continuously: even at severe corruption the decision boundary does not collapse completely.

In conclusion, the SNN leverages representation learning to remain robust; the fuzzy model is competitive for smooth noise but fails on discontinuities.

4.3 Effect of preprocessing (inference–only run)

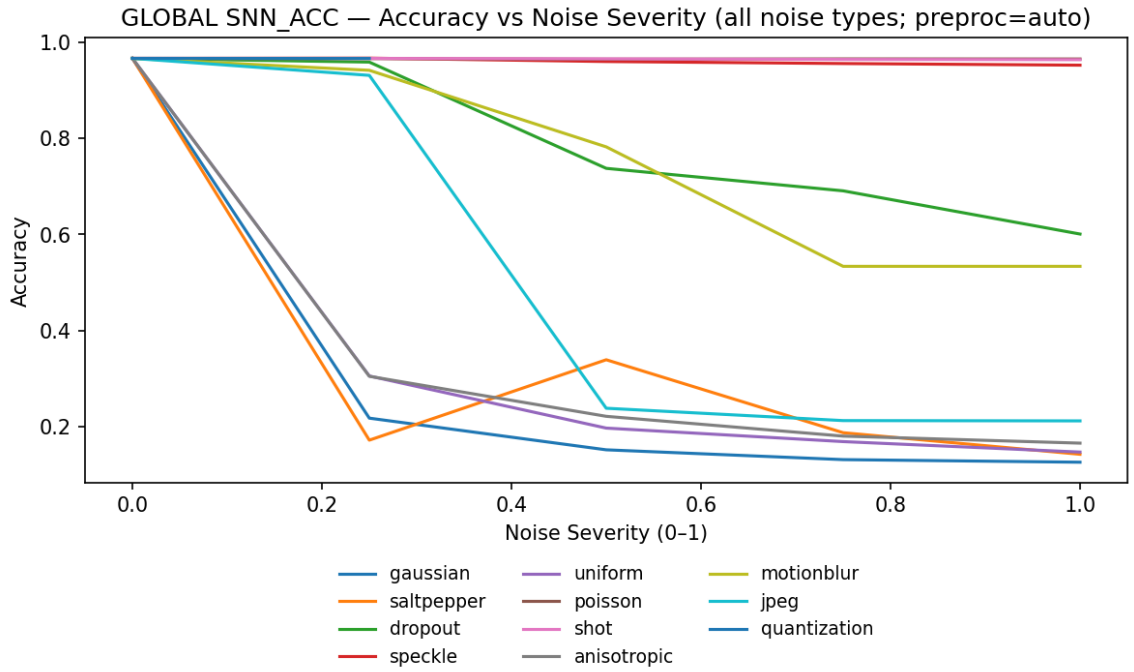

Figure 3: SNN with "auto" preprocessing, severity gate = 0.5 (older run; inference only).

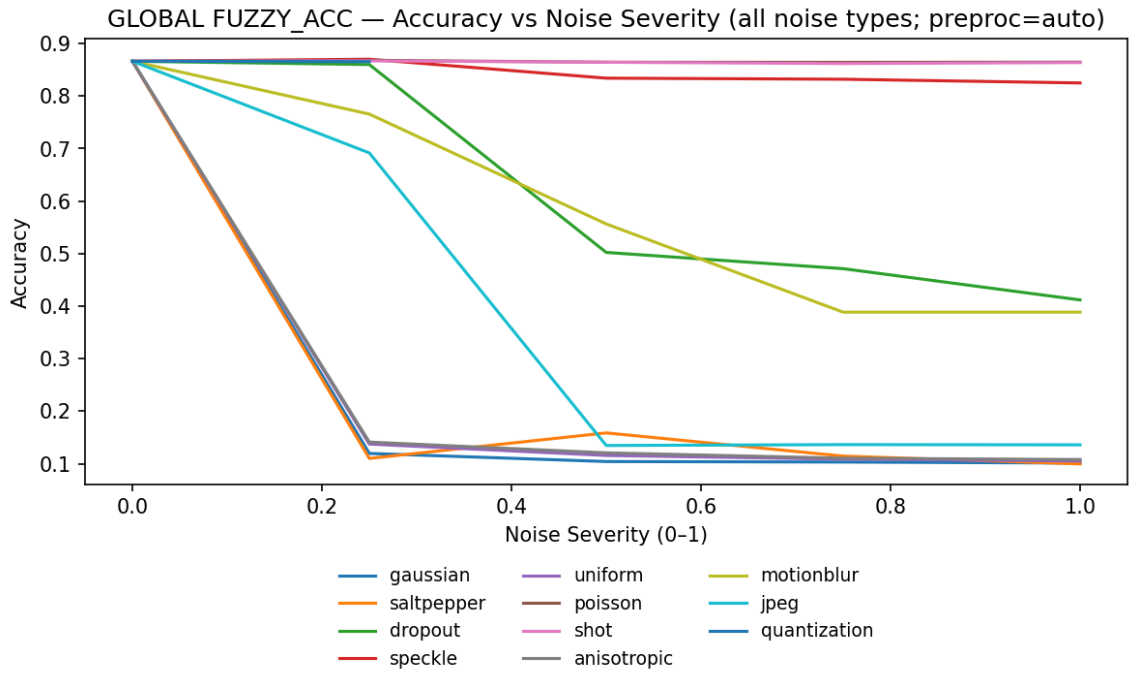

Figure 4: Fuzzy with "auto" preprocessing, **severity gate** = 0.5 (older run; inference only).

What changes here Figures 3–4 repeat the sweep but enable an auto preprocessing heuristic that activates only for severities ≥ 0.5 (to avoid unnecessary smoothing at low severities). In this older run the policy is uniform once the gate is open (no per-point validation), so the curves are smooth—there are no flat or stopped segments. We observe mild recovery for structured

distortions (e.g., motion blur, JPEG), while salt–pepper and high-variance Gaussian remain difficult: classical denoisers remove noise *and* fine strokes.

Inference-only takeaway: gating avoids hurting low-severity cases, and some noise types benefit slightly at high severity, but the overall gap persists because models were trained on clean strokes while the test inputs are filtered—i.e., a train–test distribution mismatch.

Motivation for a second stage (train–time alignment): The inference-only setup reveals that beyond moderate severity (≥ 0.5) most denoisers also remove class-informative structure. Even when the image looks "cleaner", the statistics differ from the clean training set. To address this, Section 4.4 introduces a second setting where the same auto preprocessing is also applied during training (with noise augmentation) so that PCA, fuzzy, and SNN learn on the same filtered statistics they will see at test time.

4.4 Effect of preprocessing with train–time alignment

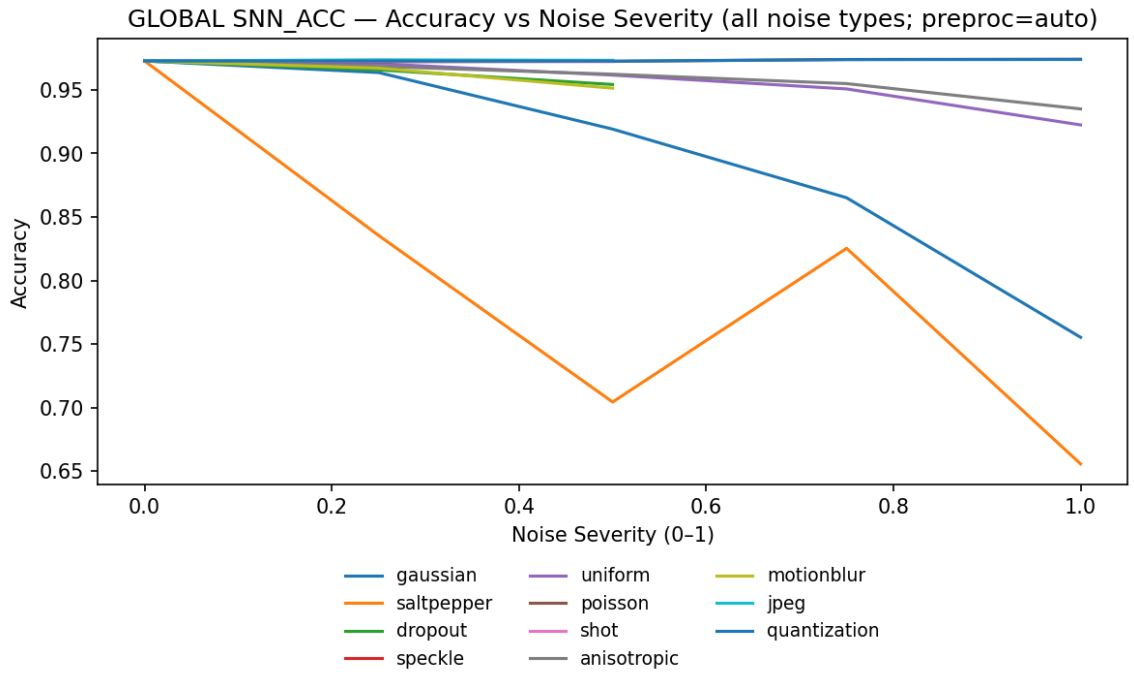

Figure 5: SNN with auto preprocessing and train-time alignment. A validation gate now applies auto only when it outperforms "none" at that (noise, severity).

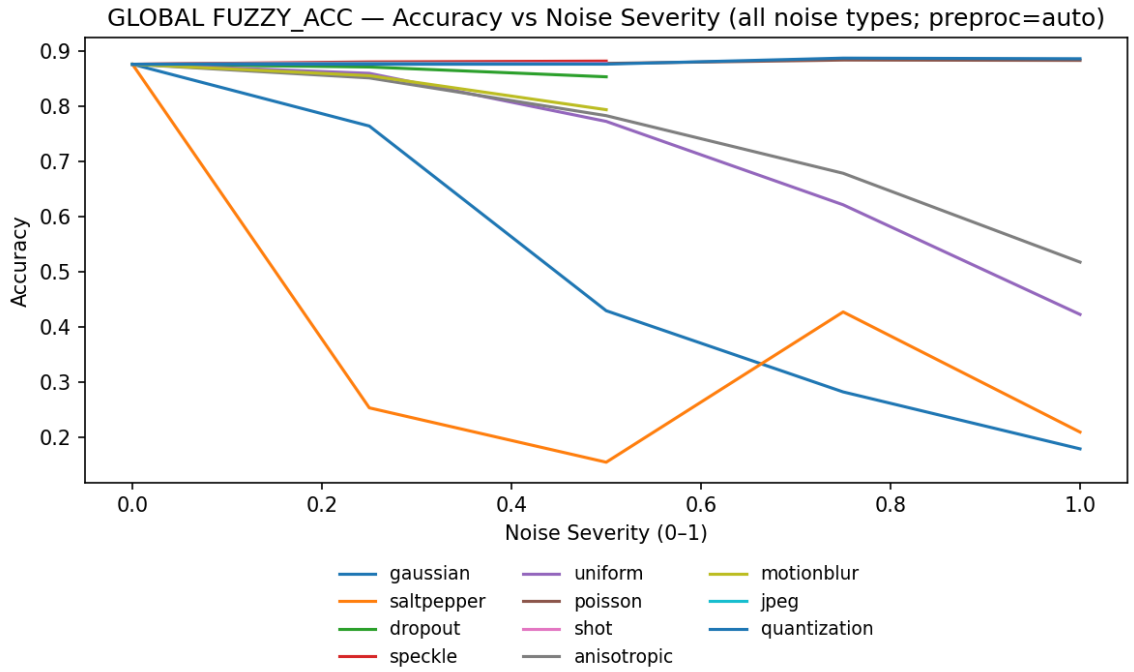

Figure 6: Fuzzy with auto preprocessing *and* train-time alignment. Same validation gate as Fig. 5.

Why curves can "flatten" here. In the aligned runs we add a validation gate: for each (noise, severity) we keep auto only if it beats the unfiltered baseline; otherwise we fall back to none. This yields threshold-like behaviour: some noise types stop receiving auto beyond roughly 0.5 because denoising would remove useful stroke detail. Those points then ride the baseline, which appears as flat/paused segments. Conversely, distortions like motion blur or quantization still benefit from filtering and continue to improve or degrade more slowly beyond the gate.

Aligned takeaway: once training and testing share the same filtered statistics, the SNN shows more stable mid-severity performance and small gains for blur/JPEG. Heavy salt–pepper and strong Gaussian remain fundamental failure modes for both models because class information is irrevocably destroyed.

4.5 Per-noise comparisons

Table 2: Salt–pepper (mid/high severity): effect of preprocessing.

Model	$s = 0.50$	$s = 0.75$
Fuzzy + none	0.120	0.114
Fuzzy + median	0.228	0.334
SNN + none	0.648	0.502
SNN + median	**0.857**	**0.823**

Salt–pepper as a model–dependent failure mode. Salt–pepper behaves in two qualitatively different ways depending on the model. For the fuzzy classifier, preprocessing brings only a modest gain (from ≈ 0.12 to ≈ 0.33 at s=0.75), because the membership functions operate on centroid structure that is already lost once strokes are deleted. In contrast, the SNN benefits dramatically: median filtering at s=0.50 boosts accuracy from ≈ 0.65 to ≈ 0.86, and

from ≈ 0.50 to ≈ 0.82 at s=0.75. The network can still exploit the smoothed silhouette to reconstruct discriminative features, even if some local pixels are missing.

This shows that preprocessing is not uniformly useful or useless: it depends on whether the downstream model knows how to exploit the denoised structure. For fuzzy models, missing strokes are unrecoverable; for neural models, partial geometric recovery is still enough to anchor the representation in feature space.

5 Confusion matrices

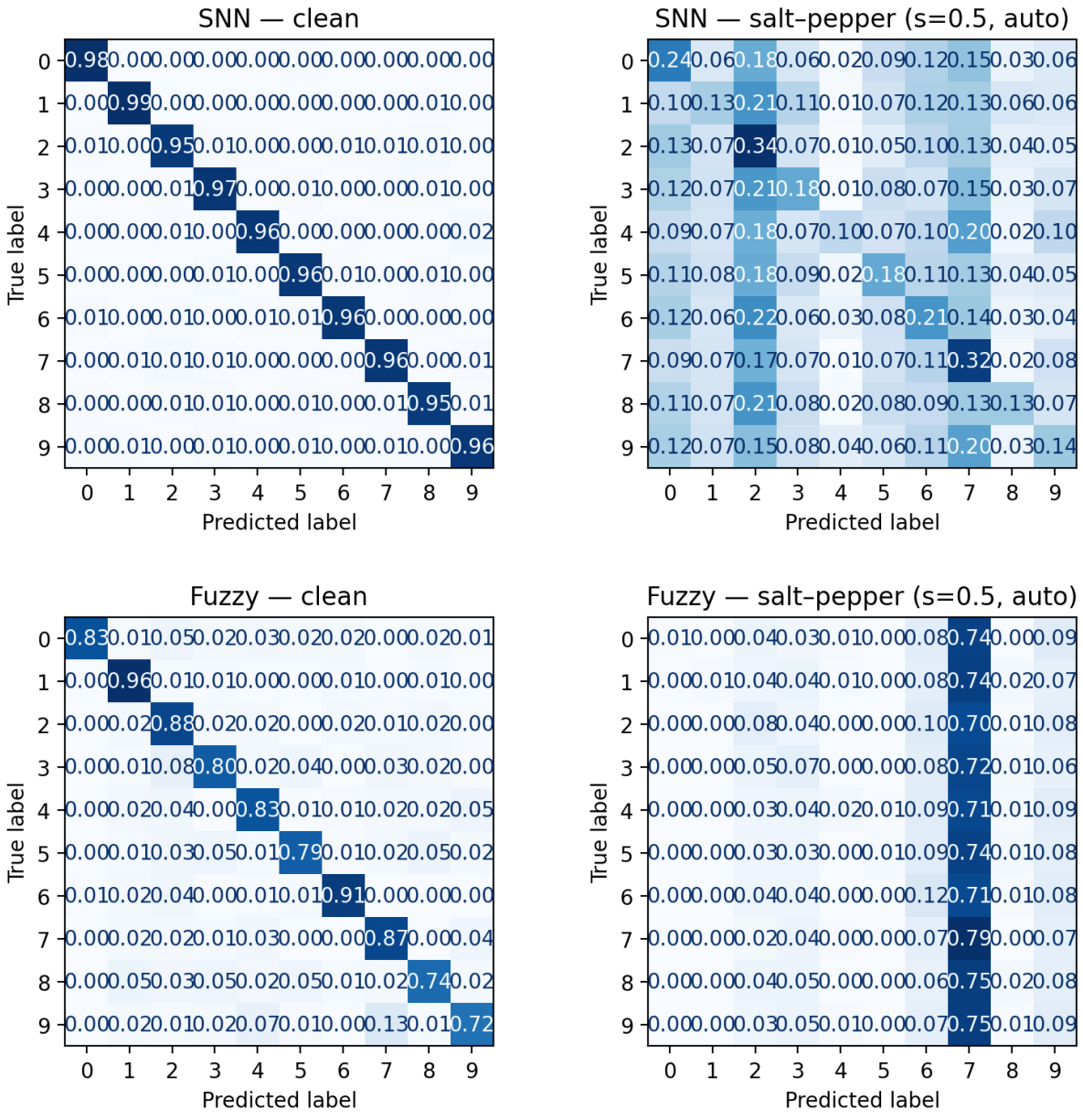

Figure 7: Confusion matrices (rows = true class, columns = predicted class). Fuzzy collapses onto a single dominant prediction under corruption, whereas SNN distributes its errors across geometrically similar digits.

6 Confusion matrices

Figure 7 compares clean and salt–pepper (s=0.5) confusion matrices. Two effects are visible.

First, under severe salt–pepper the **fuzzy model collapses almost entirely onto a single surrogate class** (here class "7"): once strokes are broken into disconnected pixel islands, the Gaussian memberships no longer activate correctly, so the linear readout picks the class whose centroid is closest in low-level statistics rather than geometry. This is consistent with the near-flat accuracy observed in the global curves: the model is not making "noisy" decisions, but systematically mapping many digits to the same one.

Second, the SNN does degrade strongly, but its confusion is **distributed across multiple look-alike digits**: thin or open strokes (1, 7, 2, 3, 8) become mutually confusable, rather than all collapsing into a single attractor. This indicates that a learned representation preserves some intra-class structure even after heavy corruption, whereas the fuzzy model effectively loses its class separability once membership functions no longer align with the corrupted inputs.

7 Conclusion

Q1: How do the models degrade? Across the standardized severity sweep, the SNN loses accuracy slowly and remains usable under most smooth distortions (Poisson, uniform, speckle). The fuzzy model is competitive for smooth noise at low severity but collapses under discontinuous corruptions (salt–pepper), where missing strokes push samples outside its rule regions.

Q2: Does lightweight preprocessing help? It depends on the noise and the model. Median filtering is highly effective for salt–pepper on the SNN (e.g., at s=0.5 the SNN improves from ~ 0.65 to ~ 0.86), but brings only modest gains for the fuzzy model—once strokes are gone, centroid-based rules cannot "reconstruct" the geometry. For smooth/compression-like distortions (e.g., JPEG, quantization), gentle denoisers sometimes stabilize accuracy but never fully recover the clean baseline.

What mattered most. A severity gate avoids hurting low-noise cases, and aligning train and test distributions (by applying the same preprocessing during training) is more important than using a stronger denoiser at test time. Taken together, these findings suggest a practical recipe: use a simple SNN with a severity-gated preprocessor, and—if high-severity noise is expected—include the same preprocessing in the training pipeline. Fuzzy rules remain valuable for interpretability and for smooth noise, but their brittleness to sparse, discontinuous corruptions is the main limitation.

For the fuzzy model, most of the achievable robustness does not come from the denoiser itself, but from reducing train–test mismatch. Multiple rules per class and a linear readout already make the fuzzy baseline much stronger than a single–rule fuzzy classifier, but preprocessing only helps when the model was trained on data that has been filtered in the same way. In other words, the main wins come from matching distributions, not from stronger filtering.

GitHub link

Public repository: https://github.com/Al3c2/IS-Project.git
Full result tables in pdf form and csv files for raw data from results all provided in the repo.